

Workflow Fundamentals

Series: WFLOW | **Notebook:** 1 of 9 | **Created:** January 2026

Introduction to Dynatrace Workflows

Dynatrace Workflows is the automation engine that enables event-driven automation, scheduled tasks, and integration orchestration. This notebook introduces core concepts, components, and your first workflow.

Table of Contents

1. What Are Workflows?
 2. Workflows vs Other Automation
 3. Workflow Components
 4. Accessing Workflows
 5. Execution Model
 6. Your First Workflow
 7. Viewing Execution History
 8. Next Steps
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	<code>automation:workflows:read</code> , <code>automation:workflows:write</code>
Prior Knowledge	Basic Dynatrace navigation

1. What Are Workflows?

Workflows are automated sequences of tasks that execute in response to events, schedules, or manual triggers. They enable:

Capability	Example
Alert Notifications	Send Slack/Teams messages when problems occur
Incident Management	Create PagerDuty/ServiceNow tickets automatically
Auto-Remediation	Restart pods, scale resources, run scripts
Reporting	Generate and email daily reports
Integration	Sync data with external systems
Data Enrichment	Query APIs and enrich events

Key Benefits

- **No Infrastructure** - Runs in Dynatrace, no external servers needed
- **Event-Driven** - Triggers on Davis problems, metric events, schedules
- **Low-Code + Code** - Visual builder with JavaScript option
- **Secure** - Built-in secrets management, RBAC, audit logs
- **Observable** - Execution history, metrics, debugging

2. Workflows vs Other Automation

Dynatrace offers multiple automation approaches. When should you use Workflows?

Approach	Best For	Limitations
Workflows	Event-driven automation, notifications, integrations	Rate limits, execution time limits
Alerting Profiles (Legacy)	Simple email/webhook notifications	Limited routing, no logic
Apps (AppEngine)	Custom UI, complex applications	Requires development expertise
Extensions	Data collection, custom metrics	Not for automation/notifications
Site Reliability Guardian	Release validation, SLO verification	Specific to deployment validation

Migration from Legacy Alerting

Legacy Alerting	Workflow Equivalent
Problem notification → Email	Davis Problem trigger → Email task
Problem notification → Webhook	Davis Problem trigger → HTTP Request task
Custom integration	Davis Problem trigger → JavaScript + HTTP

Recommendation: New implementations should use Workflows. Legacy alerting profiles will eventually be deprecated.

3. Workflow Components

Every workflow consists of these components:

3.1 Trigger

What starts the workflow:

Trigger Type	Starts When	Use Case
Davis Problem	Davis AI detects a problem	Alert notifications
Davis Event	Metric threshold breached	Capacity alerts
Schedule	Cron expression matches	Daily reports
On-Demand	Manual execution or API call	Testing, ad-hoc runs
Event Trigger	Custom/business event ingested	Business process automation

3.2 Tasks

Actions the workflow performs:

Task Category	Examples
Notifications	Slack, Teams, Email, PagerDuty
Incident Management	ServiceNow, Jira
Data Queries	DQL queries, entity lookups
HTTP Requests	REST API calls to any endpoint
JavaScript	Custom code execution
Control Flow	Wait, conditions, loops

3.3 Conditions

Logic that controls task execution:

```
conditions:  
  - name: is_critical  
    expression: '{{ event()["severity"] == "CRITICAL" }}'
```

3.4 Expressions

Dynamic values using Jinja2 syntax:

```
{{ event()["title"] }}          # Access trigger data
{{ result("task_name").output }} # Access task output
{{ env.SECRET_NAME }}           # Access secrets
```

4. Accessing Workflows

Navigation

1. Open Dynatrace
2. Go to **Apps** in the left navigation
3. Search for and open **Workflows**

Or use the direct URL: `https://<your-environment>/ui/apps/dynatrace.automations/workflows`

Required Permissions

Permission	Allows
<code>automation:workflows:read</code>	View workflows, view execution history
<code>automation:workflows:write</code>	Create, edit, delete workflows
<code>automation:workflows:run</code>	Execute workflows manually
<code>automation:workflows:admin</code>	Full admin access

Workflow Listing

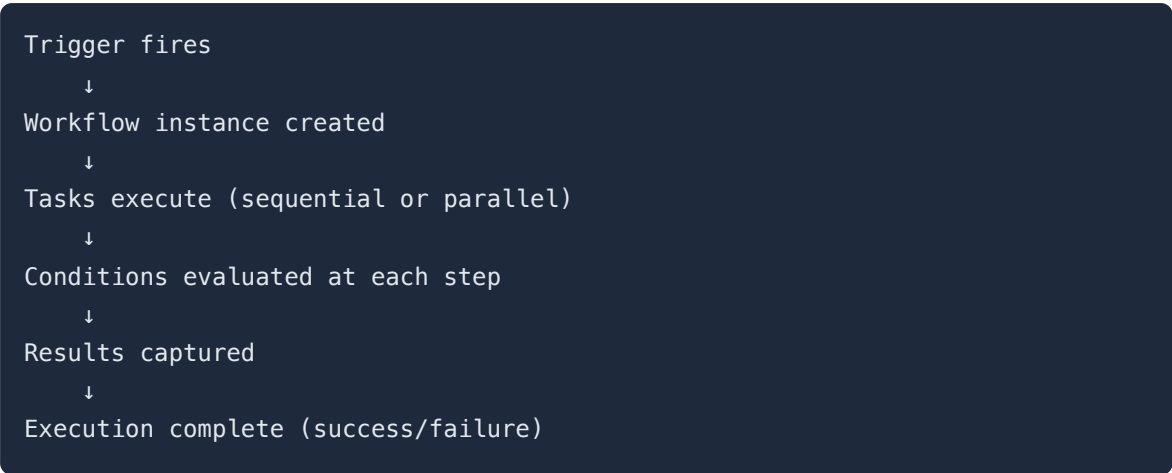
The Workflows app shows:

- **My Workflows** - Workflows you created
- **Shared with me** - Workflows shared by others
- **All Workflows** - All workflows you can access

5. Execution Model

Understanding how workflows execute is important for designing reliable automation.

Execution Flow



Limits and Quotas

Limit	Value	Notes
Max execution time	15 minutes	Per workflow execution
Max concurrent executions	100	Per environment
Max tasks per workflow	50	Design for simplicity
Task timeout	5 minutes	Per individual task
Rate limit	Varies	Depends on subscription

Execution States

State	Meaning
<code>RUNNING</code>	Currently executing
<code>SUCCEEDED</code>	Completed successfully
<code>FAILED</code>	One or more tasks failed
<code>CANCELLED</code>	Manually cancelled
<code>TIMED_OUT</code>	Exceeded execution limit

6. Your First Workflow

Let's create a simple workflow that logs a message when manually triggered.

Step 1: Create New Workflow

1. Open **Workflows** app
2. Click **+ Workflow**
3. Name it: `Hello World Workflow`

Step 2: Configure Trigger

1. Click the trigger box (top of canvas)
2. Select **On demand** trigger
3. This allows manual execution for testing

Step 3: Add a Task

1. Click **+** below the trigger
2. Search for **Run JavaScript**
3. Add the following code:

```
export default async function() {
  const now = new Date().toISOString();
  console.log(`Hello from Dynatrace Workflows at ${now}`);

  return {
    message: "Workflow executed successfully!",
    timestamp: now
  };
}
```

Step 4: Save and Run

1. Click **Save**
2. Click **Run** (play button)
3. View execution results

Expected Output

The task output should show:

```
{
  "message": "Workflow executed successfully!",
  "timestamp": "2026-01-27T12:00:00.000Z"
}
```

7. Viewing Execution History

Monitor your workflow executions using DQL queries.

```
// Recent workflow executions
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| fields timestamp,
          workflow.name,
          execution.id,
          execution.status,
          execution.duration
| sort timestamp desc
| limit 20
```



```
// Workflow execution summary
fetch events, from: now() - 7d
| filter event.type == "automation.workflow.execution"
| summarize
    total = count(),
    succeeded = countIf(execution.status == "SUCCEEDED"),
    failed = countIf(execution.status == "FAILED"),
    by:{workflow.name}
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
| sort total desc
| limit 20
```

```
// Failed workflow executions
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| filter execution.status == "FAILED"
| fields timestamp, workflow.name, execution.id, execution.error
| sort timestamp desc
| limit 20
```

Next Steps

Now that you understand workflow fundamentals, continue with:

Recommended Path

1. **WFLOW-02: Triggers & Event Types** - Configure event-driven triggers
2. **WFLOW-03: Alert Notification Basics** - Send Slack, Teams, email alerts
3. **WFLOW-04: Advanced Notification Routing** - Conditional routing and escalation

Key Takeaways

- Workflows automate responses to events, schedules, and manual triggers
 - Components: Triggers → Tasks → Conditions → Results
 - Use Workflows for notifications, integrations, and auto-remediation
 - Monitor execution history via DQL queries
-

Summary

In this notebook, you learned:

- What Dynatrace Workflows are and when to use them
 - How workflows compare to other automation options
 - Core workflow components (triggers, tasks, conditions)
 - How to access and navigate the Workflows app
 - The execution model and limits
 - How to create your first workflow
 - How to query execution history
-

References

- [Dynatrace Workflows Documentation](#)
 - [Workflow Triggers](#)
 - [Workflow Actions](#)
 - [Jinja Expressions](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.